Lab 4 Solution: Simple Python Database Connections

@sidraliaqat

Student Details

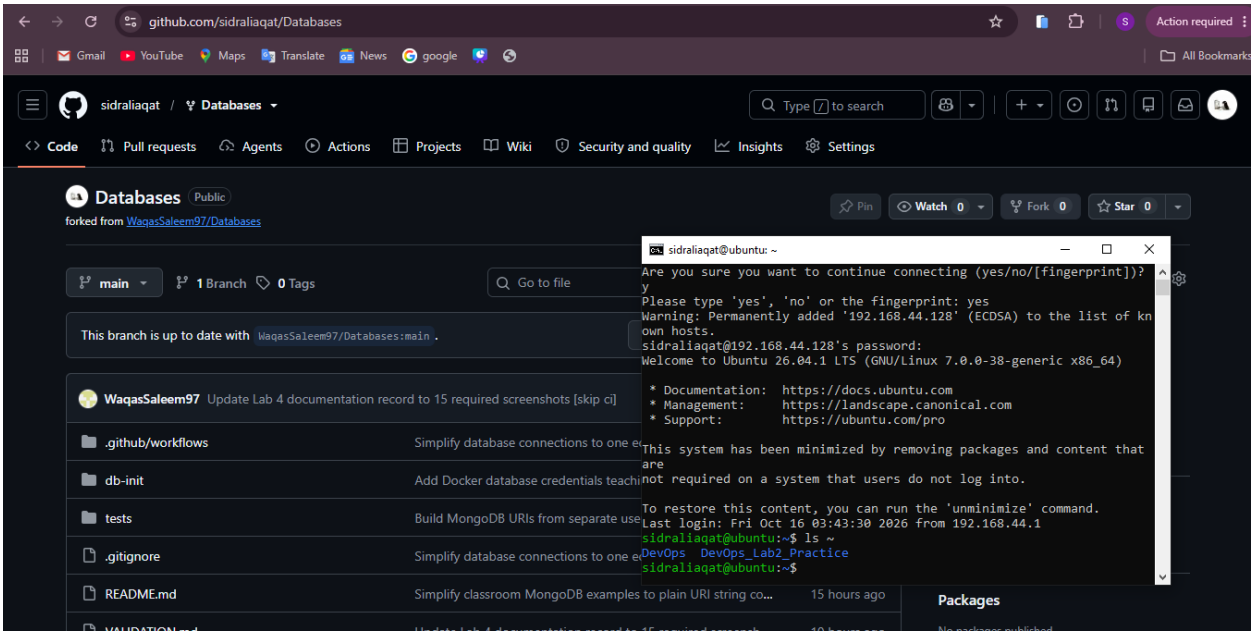
Field	Value
Name	Sidra Liaqat
GitHub username	sidraliaqat
Ubuntu prompt	sidraliaqat@ubuntu
Databases fork URL	https://github.com/sidraliaqat/Databases
DevOps Lab04 URL	https://github.com/sidraliaqat/DevOps/tree/main/Labs/Lab04

Every screenshot below must show your username@ubuntu prompt, the command, and its output.

Task 1: Fork and Clone the Databases Repository

I forked the Databases repository on GitHub, kept the name Databases, and cloned my own fork to the Ubuntu server with `git clone`. The fork page shows my username and the original repository as its source.

Screenshot 1: databases_fork.png (split screen, GitHub page and terminal)



Screenshot 2: databases_clone.png

```
sidraliaqat@ubuntu:~$ ls ~
DevOps  DevOps_Lab2_Practice
sidraliaqat@ubuntu:~$ cd ~
sidraliaqat/Databases.gitsidraliaqat@ubuntu:~$ git clone git@github.com:sidraliaqat/Databases.git
Cloning into 'Databases'...
remote: Enumerating objects: 101, done.
remote: Counting objects: 100% (101/101), done.
remote: Compressing objects: 100% (90/90), done.
remote: Total 101 (delta 30), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (101/101), 39.86 KiB | 7.97 MiB/s, done.
Resolving deltas: 100% (30/30), done.
sidraliaqat@ubuntu:~$ cd ~/Databases
sidraliaqat@ubuntu:~/Databases$ ls
README.md      credentials.ini.example  requirements.txt
VALIDATION.md  db-init                  tests
compose.yaml   db_connect.py
sidraliaqat@ubuntu:~/Databases$
```

Observation: (write one or two sentences on what you saw)

Task 2: Install Docker, Docker Compose, and Python

I installed `docker.io` and `curl`, downloaded the Docker Compose plug-in, added my account to the docker group, signed out completely, and reconnected. Then I checked `git --version`, `docker --version` and `docker compose version`. I also created a virtual environment in `~/Databases` and installed the packages from `requirements.txt`.

Screenshot: docker_installation.png (three commands with their versions)

```
sidraliaqat@ubuntu:~$ sudo mkdir -p /usr/local/lib/docker/cli-plugins
sidraliaqat@ubuntu:~$ sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-c
ompose
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0      0   0      0   0      0      0      0           00:01              0
  0      0   0      0   0      0      0      0           00:01              0
100 31.16M 100 31.16M   0      0 700.1k      0   00:45   00:45          1.12M
sidraliaqat@ubuntu:~$ sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
sidraliaqat@ubuntu:~$ git --version
ersion
docker compose versiongit version 2.53.0
sidraliaqat@ubuntu:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
sidraliaqat@ubuntu:~$ docker compose version
Docker Compose version v5.6.1
sidraliaqat@ubuntu:~$
```

Versions installed: Git (version), Docker (version), Docker Compose (version)

Task 3: Start the Database Services

I started MySQL and the three MongoDB containers with `docker compose up -d --wait`, then checked them with `docker compose ps` and `docker compose port`.

Screenshot 1: database_services.png (all four services healthy)

```
sidraliaqat@ubuntu:~/Databases$ docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod
[+] up 4/4
 ✔ Container databases-mongo-test-1 Healthy                                                                    0.6s
 ✔ Container databases-mongo-prod-1 Healthy                                                                    0.6s
 ✔ Container databases-mysql-1      Healthy                                                                    0.6s
 ✔ Container databases-mongo-dev-1  Healthy                                                                    0.6s
sidraliaqat@ubuntu:~/Databases$ docker compose ps
NAME                    IMAGE       COMMAND                  SERVICE      CREATED          STATUS                    PORTS
databases-mongo-dev-1   mongo:7.0   "docker-entrypoint.s…"   mongo-dev    2 minutes ago    Up 2 minutes (healthy)    127.0.0.1:27017-
>27017/tcp
databases-mongo-prod-1  mongo:7.0   "docker-entrypoint.s…"   mongo-prod   2 minutes ago    Up 2 minutes (healthy)    127.0.0.1:27019-
>27017/tcp
databases-mongo-test-1  mongo:7.0   "docker-entrypoint.s…"   mongo-test   2 minutes ago    Up 2 minutes (healthy)    127.0.0.1:27018-
>27017/tcp
databases-mysql-1       mysql:8.4   "docker-entrypoint.s…"   mysql        32 minutes ago   Up 10 minutes (healthy)   33060/tcp, 127.0
.0.1:3307->3306/tcp
sidraliaqat@ubuntu:~/Databases$
```

Screenshot 2: database_ports.png

```
/tcp
sidraliaqat@ubuntu:~/Databases$ ss -lntp | grep -E ':3307|:27017|:27018|:27019'
LISTEN 0      4096       127.0.0.1:27019      0.0.0.0:*
LISTEN 0      4096       127.0.0.1:27018      0.0.0.0:*
LISTEN 0      4096       127.0.0.1:27017      0.0.0.0:*
LISTEN 0      4096       127.0.0.1:3307       0.0.0.0:*
sidraliaqat@ubuntu:~/Databases$
```

Database	Host endpoint	Database name
MySQL	127.0.0.1:3307	school_mysql
MongoDB dev	127.0.0.1:27017	school_dev
MongoDB test	127.0.0.1:27018	school_test
MongoDB prod (simulated)	127.0.0.1:27019	school_prod

Task 4: MySQL with Values in the Code

I ran `python db_connect.py` with the MySQL settings written directly in the file. The program connected to `school_mysql` on 127.0.0.1 port 3307 and read the environment value from the `environment_info` table.

Screenshot: mysql_hardcoded_connection.png (shows `Connected to MySQL: school_mysql`, server `127.0.0.1 3307`, and `Environment: dev`)

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
(.venv) sidraliaqat@ubuntu:~/Databases$ cd ~/Databases
(.venv) sidraliaqat@ubuntu:~/Databases$ nano db_connect.py
(.venv) sidraliaqat@ubuntu:~/Databases$ python db_connect.py
Connected to MySQL: school_mysql
Server: 127.0.0.1 3307
Environment: dev
(.venv) sidraliaqat@ubuntu:~/Databases$
```

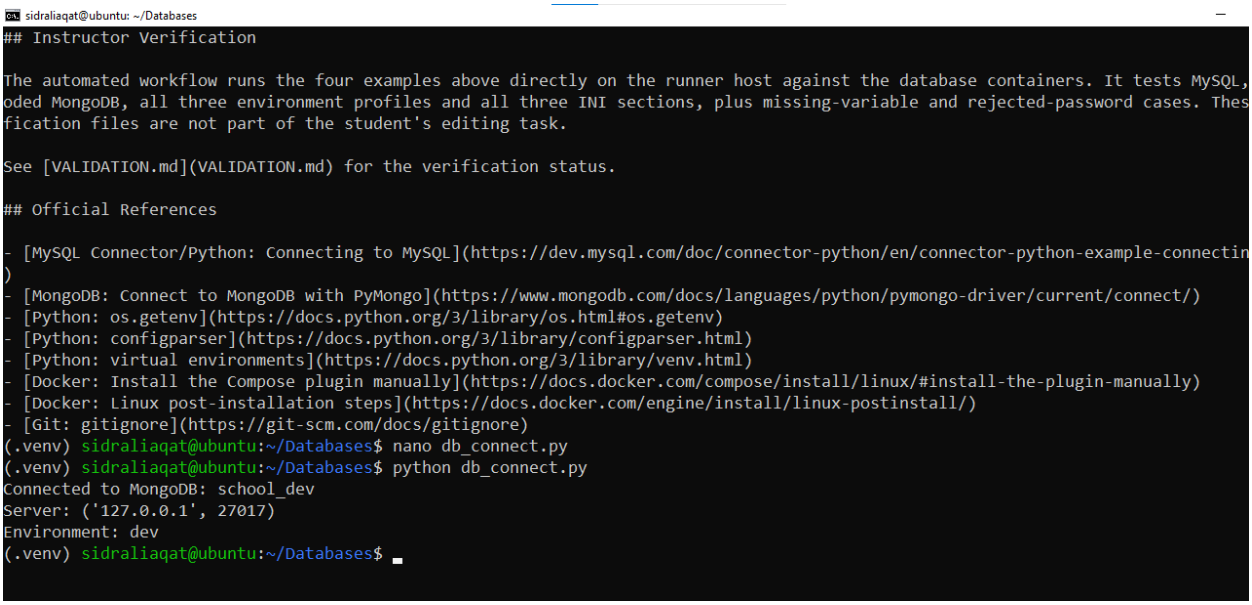
Result: (copy the three output lines in your own words)

What each setting does: `user` and `password` are the credentials, `host` and `port` locate the MySQL service, and `database` picks the database.

Task 5: MongoDB with Values in the Code

I replaced the contents of `db_connect.py` with the MongoDB version and ran the same `python db_connect.py` command. The program joined the username, password, endpoint and database name into a URI, sent a `ping`, and printed the database name, server address and stored environment.

Screenshot: mongo_hardcoded_connection.png (shows database `school_dev`, port 27017, environment `dev`)

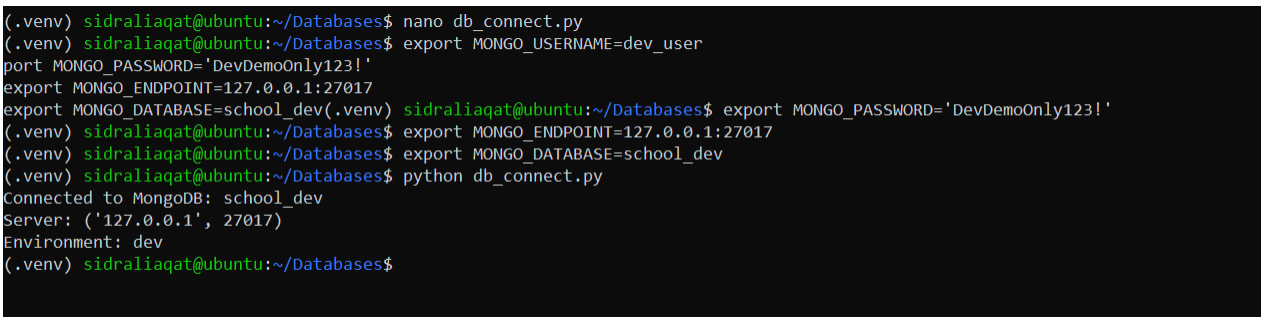


Why `ping` matters: creating a client or selecting a database does not contact the server. `database.command("ping")` sends a real request, so it proves the connection works.

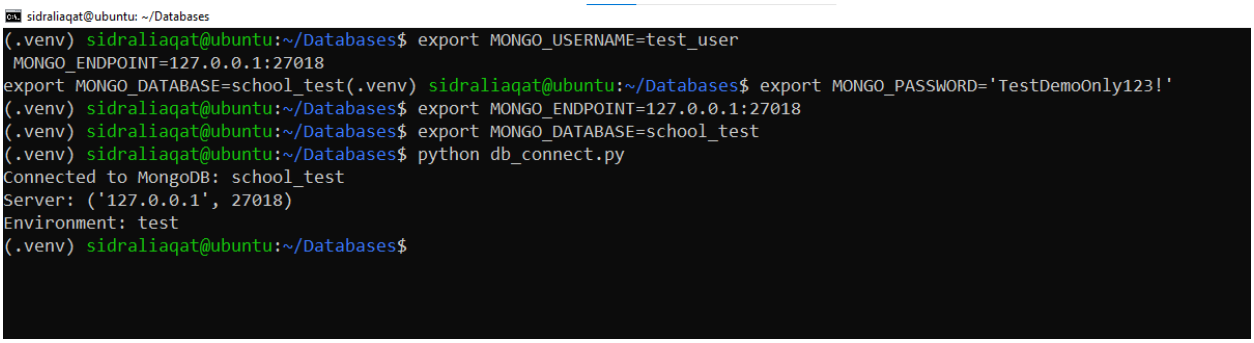
Task 6: MongoDB with os.getenv()

The code now reads `MONGO_USERNAME`, `MONGO_PASSWORD`, `MONGO_ENDPOINT` and `MONGO_DATABASE` from the shell with `os.getenv()`. I exported a matching set of four values for each environment, ran `clear`, then ran the unchanged program. The exports and the password are not shown in any screenshot.

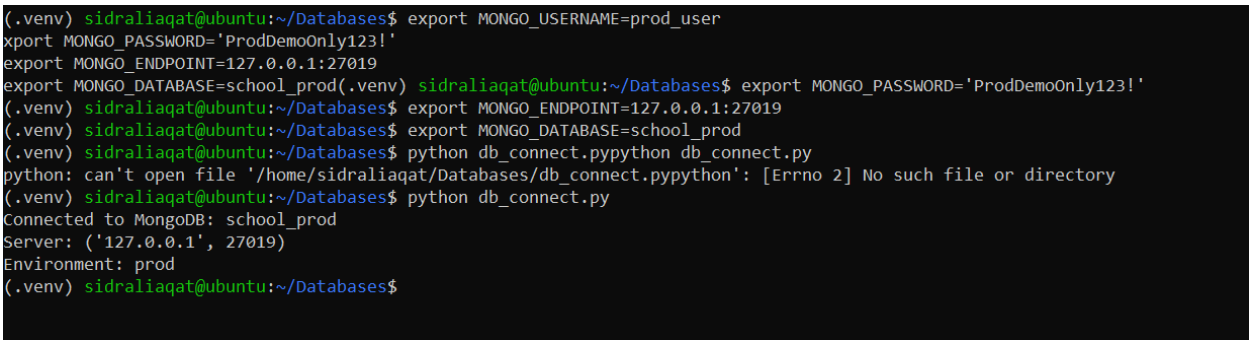
Screenshot 1: mongo_env_dev.png (`school_dev`, port 27017, environment `dev`)



Screenshot 2: mongo_env_test.png (`school_test`, port 27018, environment `test`)



Screenshot 3: mongo_env_prod.png (`school_prod`, port 27019, environment `prod`)

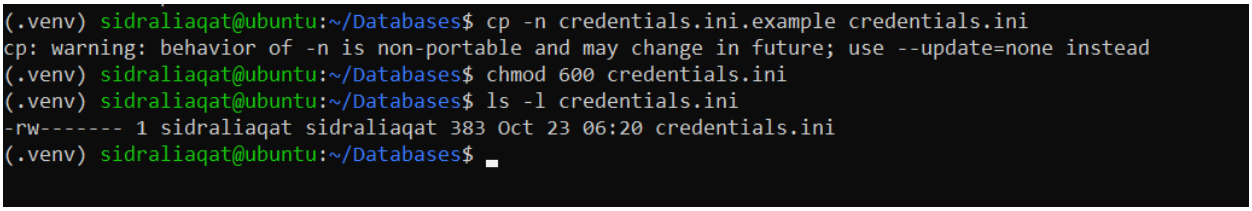


Observation: the same program connected to three different databases because only the process environment changed.

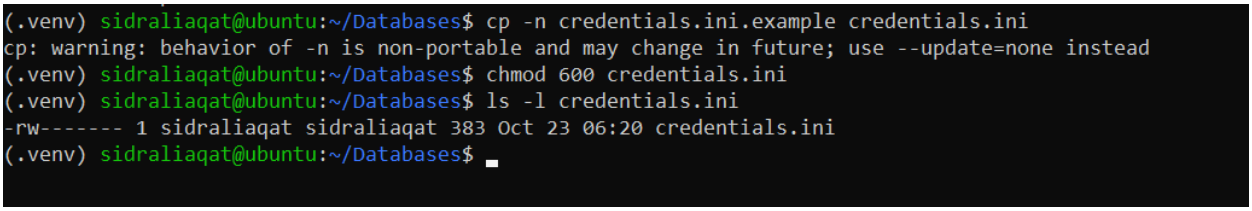
Task 7: MongoDB with a Credentials File

I copied `credentials.ini.example` to `credentials.ini`, restricted it with `chmod 600`, and changed `db_connect.py` to read the settings with `configparser`. The `APP_ENV` variable picks the section (`dev`, `test` or `prod`), with `dev` as the default. The file is ignored by Git and was not committed.

Screenshot 1: credential_files_created.png (output of `ls credentials.ini`)



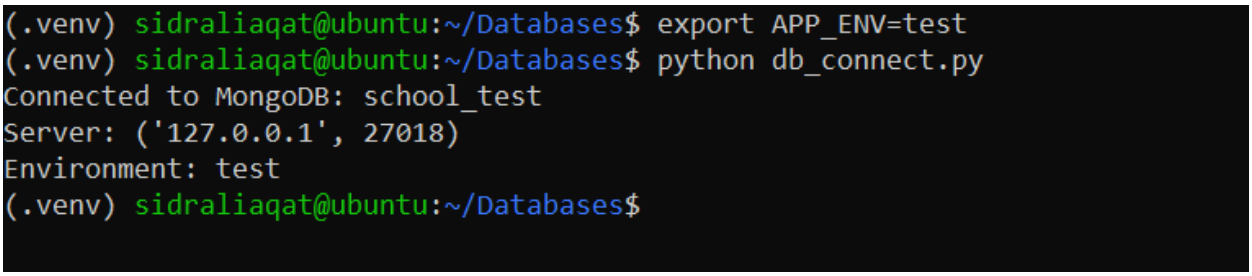
Screenshot 2: credential_file_permissions.png (shows `600 credentials.ini`)



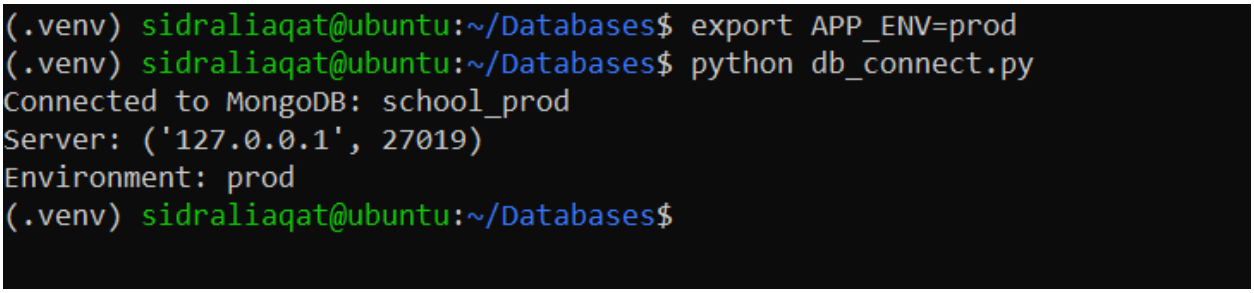
Screenshot 3: mongo_file_dev.png (`APP_ENV=dev`)



Screenshot 4: mongo_file_test.png (`APP_ENV=test`)



Screenshot 5: mongo_file_prod.png (`APP_ENV=prod`)



Observation: the connection block stayed the same; only the source of the settings changed from shell variables to a file.

Summary

I edited one Python file, `db_connect.py`, through four connection examples: MySQL with values in the code, MongoDB with values in the code, MongoDB with `os.getenv()`, and MongoDB with a credentials file. Each run was checked using the database name, the endpoint and the environment value stored in the database. Python ran directly on Ubuntu and Docker ran only the databases.